

Implementation of the LDG Method in Exasim

1. Summary

Exasim implements the LDG method by rewriting the PDE into a first-order system, recovering the auxiliary gradient variable q locally, then assembling the u residual from element-volume and face-flux terms. The core LDG path is in:

- `backend/Discretization/residual.cpp`
- `backend/Discretization/qresidual.cpp`
- `backend/Discretization/uresidual.cpp`
- `backend/Discretization/getuhat.cpp`
- `backend/Discretization/geometry.cpp`

The PDE-specific terms in the weak formulation are injected through the model-driver layer in:

- `backend/Model/ModelDispatch/model_drivers_abi.cpp`

2. Assumptions

- I am describing the LDG residual implementation, not the HDG global trace-system implementation.
- The PDE is written in first-order form with unknowns u and q , where q represents gradients or gradient-like auxiliary variables.
- I use the Exasim code sign conventions as implemented in the residual assembly. These are equivalent to standard LDG weak forms, but the algebraic residual may be written with the opposite overall sign from a textbook presentation.
- Arrays are stored in Exasim's flattened column-major style and are evaluated by:
 - gather nodal data
 - interpolate to Gauss points
 - evaluate model terms
 - apply metric/Jacobian factors
 - project back to nodal residuals
- w in Exasim may also denote an additional algebraic/auxiliary field `wdg`; that is separate from the usual test function notation. Below I will use v and ϕ for test functions.

3. Design reasoning

A standard LDG formulation for a diffusion-type or mixed first-order PDE is:

$$\begin{aligned} q - \nabla u &= 0, \\ u_t + \nabla \cdot F(u, q, o, w) &= S(u, q, o, w). \end{aligned}$$

A typical elementwise weak form is:

- q equation:

$$(q, v)_K + (u, \nabla \cdot v)_K - \langle \hat{u}, v \cdot n \rangle_{\partial K} = 0$$

- u equation:

$$(u_t, \phi)_K + (S, \phi)_K + (F, \nabla \phi)_K - \langle \hat{f}, \phi \rangle_{\partial K} = 0$$

Exasim implements each of these terms explicitly.

A. High-level LDG execution order

For the u residual, Exasim does the following in `backend/Discretization/residual.cpp`:

1. `GetUhat(...)`
2. `GetQ(...)` if `common.ncq > 0`
3. `GetW(...)` if `common.ncw > 0`
4. `RuElem(...)`
5. `RuFace(...)`

So the LDG structure is:

- build face trace/boundary states
- recover local gradient variable q
- optionally recover algebraic field w
- assemble the u residual

That is the core LDG implementation pattern in Exasim.

B. Implementation of the weak q equation

This is handled in `backend/Discretization/qresidual.cpp`.

Term 1: $(u, \nabla \cdot v)_K$ Implemented in `RqElemBlock(...)`.

What the code does:

- gathers element nodal u from `sol.udg`
- interpolates u from nodal points to Gauss points with `Node2Gauss(...)`
- applies geometric transformation with `ApplyXx3(...)`
- projects back to element nodes with `Gauss2Node(...)`

Interpretation:

- `Node2Gauss(...)` evaluates u_h at quadrature points
- `ApplyXx3(...)` applies the mapped derivative operators through the metric tensor $\mathbf{Xx}$
- `Gauss2Node(...)` performs the quadrature-weighted test-function contraction back to the discrete residual basis

So `RqElemBlock(...)` is the implementation of the volume term $(u, \nabla \cdot v)_K$.

Term 2: $\langle \hat{u}, v \cdot n \rangle_{\partial K}$ Implemented in `RqFaceBlock(...)`.

What the code does:

- gathers face values of $\mathbf{uh}$ with `GetElemNodes(...)`
- interpolates face $\mathbf{uh}$ to face Gauss points with `Node2Gauss(...)`
- applies $\mathbf{jac} * \mathbf{n}$ with `ApplyJacNormal(...)`
- projects back to the nodal residual with `Gauss2Node(...)`

This is exactly the discrete face integral of the LDG gradient equation.

Term 3: $(q, v)_K$ Exasim does not assemble this as a separate explicit mass-matrix term in `RqElemBlock`. Instead, after building the right-hand side contributions from the volume and face terms, it applies the inverse mass matrix in `GetQ(...)` inside `backend/Discretization/residual.cpp`:

- face contributions are scattered into the element residual by `PutFaceNodes(...)`
- `ApplyMinv(...)` applies the local inverse mass matrix

This is the discrete realization of solving:

$$Mq = RHS$$

for each element, where M is the element mass matrix from $(q, v)_K$.

So in Exasim:

- $(q, v)_K$ is implemented through the local mass inverse `ApplyMinv(...)`
- $(u, \nabla \cdot v)_K$ is implemented by `RqElemBlock(...)`

- $\langle \hat{u}, v \cdot n \rangle_{\partial K}$ is implemented by `RqFaceBlock(...)`

After that, the recovered q is inserted into `sol.udg` with `ArrayInsert(...)`.
Important structural point:

- in LDG, q is local and recovered elementwise
- it is not a globally coupled trace unknown

C. Construction of $\hat{u}$ in LDG

This is handled in `backend/Discretization/getuhat.cpp`.

`GetUhat(...)` builds the face state `sol.uh`.

Interior faces For `ib == 0`, the code copies interior face data from `sol.udg` into `sol.uh`.
Interpretation:

- LDG in Exasim does not solve a global trace equation for $\hat{u}$
- $\hat{u}$ is just the face state passed into the numerical flux machinery

Boundary faces For `ib != 0`, the code:

- computes face geometry with `FaceGeom1D/2D/3D(...)`
- gathers boundary-side `udg` and `odg`
- calls `UbouDriver(...)`
- writes the boundary trace into `sol.uh`

So the boundary trace $\hat{u}$ is entirely supplied by the user/model boundary operator.
This is the implementation of the boundary data entering the LDG weak form.

D. Implementation of the weak u equation

This is handled in `backend/Discretization/uresidual.cpp`.

Term 1: time term $(u_t, \phi)_K$ In `RuElemBlock(...)`, for time-dependent problems Exasim forms a time-source-like contribution using `sol.sdg` and the time discretization factor:

- `ArrayAXPBY(...)` forms something equivalent to `sdg - dtfactor * u`
- if `common.tdfunc == 1`, `TdfuncDriver(...)` modifies this term

So the temporal weak term is implemented as part of the element source assembly rather than as a separate conceptual block.

Term 2: source term $(S, \phi)_K$ Also in `RuElemBlock(...)`:

- `SourceDriver(...)` evaluates the PDE source at Gauss points

This source is then combined into the element integrand before projection back to nodal residuals.

So $(S, \phi)_K$ is implemented by:

- Gauss-point evaluation through `SourceDriver(...)`
- element integration through `ApplyXx4(...)` and `Gauss2Node(...)`

Term 3: flux volume term $(F, \nabla \phi)_K$ Still in `RuElemBlock(...)`:

- `FluxDriver(...)` evaluates the physical flux $F(u, q, o, w)$ at Gauss points
- `ApplyXx4(...)` applies the metric/Jacobian transformation and contracts with basis gradients
- `Gauss2Node(...)` accumulates the result into the nodal residual

This is the discrete implementation of the LDG volume flux term.

So in Exasim:

- `FluxDriver(...)` gives the PDE-specific flux
- `ApplyXx4(...)` gives the mapped weak derivative action
- `Gauss2Node(...)` completes the quadrature-based residual assembly

E. Implementation of the face term $\langle \hat{f}, \phi \rangle_{\partial K}$

This is handled in `RuFaceBlock(...)` in `backend/Discretization/uresidual.cpp`.

What the code does:

- gathers `uh`, interior/exterior `udg`, and optional `wdg` on faces
- interpolates them to face Gauss points with `Node2Gauss(...)`
- for interior faces:
 - calls `FhatDriver(...)`
- for boundary faces:
 - calls `FbouDriver(...)`
- multiplies by face Jacobian with `ApplyJacFhat(...)`
- projects back with `Gauss2Node(...)`

Interpretation:

- `FhatDriver(...)` is the interior numerical flux term of LDG
- `FbouDriver(...)` is the boundary numerical flux term of LDG

This is exactly the implementation of $\langle \hat{f}, \phi \rangle_{\partial K}$.

F. How $\hat{f}$ is built

The most important PDE-dependent LDG face term is in `backend/Model/ModelDispatch/model_drivers_abi.cpp`. `FhatDriver(...)` has two modes.

Mode 1: external/provider numerical flux If `common.extFhat == 1`, Exasim directly calls:

- `abi.KokkosFhat(...)`

That means the model/provider supplies the full numerical trace flux.

Mode 2: Exasim-built LDG flux Otherwise, Exasim builds the numerical flux itself:

1. compute left and right physical fluxes via `FluxDriver(...)`
2. average them with `AverageFlux(...)`
3. form the normal component with `AverageFluxDotNormal(...)`
4. add stabilization via:
 - `abi.KokkosStab(...)` if `common.extStab >= 1`
 - or built-in `AddStabilization1/2/3(...)`

So the LDG numerical flux term is implemented as:

- average normal physical flux
- plus stabilization/jump penalty

That is the concrete code realization of the numerical flux in the weak face integral.

G. Geometry and quadrature operators

All volume and face weak terms rely on geometry computed in:

- `backend/Discretization/geometry.cpp`

`ElemGeom(...)` computes:

- physical Gauss-point coordinates `xg`
- metric terms `Xx`
- element Jacobian `jac`

`FaceGeom(...)` computes:

- face coordinates
- normals `nlg`
- face Jacobians

These are consumed by:

- `ApplyXx3(...)`
- `ApplyXx4(...)`
- `ApplyJacNormal(...)`
- `ApplyJacFhat(...)`

So every weak-form term is implemented in the standard finite-element sequence:

- reference-to-physical geometry map
- Gauss-point evaluation
- metric/Jacobian scaling
- quadrature projection back to nodal residuals

H. Optional auxiliary field w

Exasim can also carry an auxiliary variable w through `GetW(...)` in `backend/Discretization/residual.cpp`.

This is not part of the core LDG u - q structure, but it modifies source and flux evaluations when `common.ncw > 0`.

Then:

- `SourceDriver(...)` and `FluxDriver(...)` can depend on w
- the Jacobian assembly also includes w -coupling corrections

For a basic LDG description, this is an extension layer, not the essential structure.

4. Proposed implementation

The clean way to understand Exasim's LDG implementation is this mapping from weak form to code:

- q equation
 - $(q, v)_K$
 - * local mass inversion in `GetQ(...)` via `ApplyMinv(...)`
 - $(u, \nabla \cdot v)_K$
 - * `RqElemBlock(...)` in `backend/Discretization/qresidual.cpp`
 - $\langle \hat{u}, v \cdot n \rangle_{\partial K}$
 - * `RqFaceBlock(...)` in `backend/Discretization/qresidual.cpp`
- face trace construction
 - interior $\hat{u}$
 - * copied from face u in `GetUhat(...)`

- boundary $\hat{u}$
 - * `UbouDriver(...)` in `backend/Model/ModelDispatch/model_drivers_abi.cpp`
- u equation
 - $(u_t, \phi)_K$
 - * time-source assembly in `RuElemBlock(...)`
 - $(S, \phi)_K$
 - * `SourceDriver(...)` in `RuElemBlock(...)`
 - $(F, \nabla \phi)_K$
 - * `FluxDriver(...)` plus `ApplyXx4(...)` in `RuElemBlock(...)`
 - $\langle \hat{f}, \phi \rangle_{\partial K}$
 - * `FhatDriver(...)` or `FbouDriver(...)` in `RuFaceBlock(...)`
- geometry support
 - element metrics and Jacobians
 - * `ElemGeom(...)`
 - face normals and Jacobians
 - * `FaceGeom(...)`

5. Risks or numerical concerns

- The biggest source of confusion is sign convention. The textbook weak form and the algebraic residual in Exasim may differ by an overall sign, but the code comments in `qresidual.cpp` and `uresidual.cpp` make the implemented structure clear.
- u_h in LDG is not an HDG global trace unknown in Exasim. It is a face state used to construct numerical fluxes and boundary data.
- The model-driver layer is critical: the mathematical LDG terms are generic, but the actual PDE content is injected only through:
 - `FluxDriver(...)`
 - `SourceDriver(...)`
 - `FhatDriver(...)`
 - `FbouDriver(...)`
 - `UbouDriver(...)`
 - `TdfuncDriver(...)`
- If `ncw > 0`, the implementation includes extra algebraic couplings beyond the basic LDG u - q structure.
- For Jacobian-level analysis, the corresponding matrix assembly files such as `backend/Discretization/qequation` and `backend/Discretization/uequation.cpp` matter, but they are not the primary residual-level LDG path.

6. Verification plan

To verify this interpretation in the codebase:

1. Start from `backend/Discretization/residual.cpp` and trace:
 - `GetUhat(...)`
 - `GetQ(...)`
 - `RuElem(...)`
 - `RuFace(...)`
2. For the weak q equation, inspect:
 - `backend/Discretization/qresidual.cpp`
3. For the weak u equation, inspect:
 - `backend/Discretization/uresidual.cpp`
4. For the numerical flux and boundary terms, inspect:
 - `backend/Model/ModelDispatch/model_drivers_abi.cpp`
5. For a concrete PDE, test the interpretation on a simple example such as Poisson in first-order form and confirm:
 - q is recovered locally
 - u residual receives volume flux, source, and face flux terms
 - boundary data enters through `UbouDriver(...)` and `FbouDriver(...)`